CLAUDE CODE · MCP · LINEAR · CLICKUP

A task como fonte da verdade

Task tracking com agentes de IA: do diagnóstico ao deploy, num lugar só

José Daniel

O PROBLEMA

O contexto vive espalhado

Chat do agente
Some quando a sessão fecha.

Slack e threads
Pormenores importantes no problema

Design
A cargo de outro squad ou outro responsável

Documentação
Desatualiza ou atualiza com pouco feeedback

Alguém
Alguém que consegue pensar exceções

Cada sessão nova do agente começa do zero. Sem contexto, ele tende a corrigir o sintoma em vez da causa.

A PREMISSA

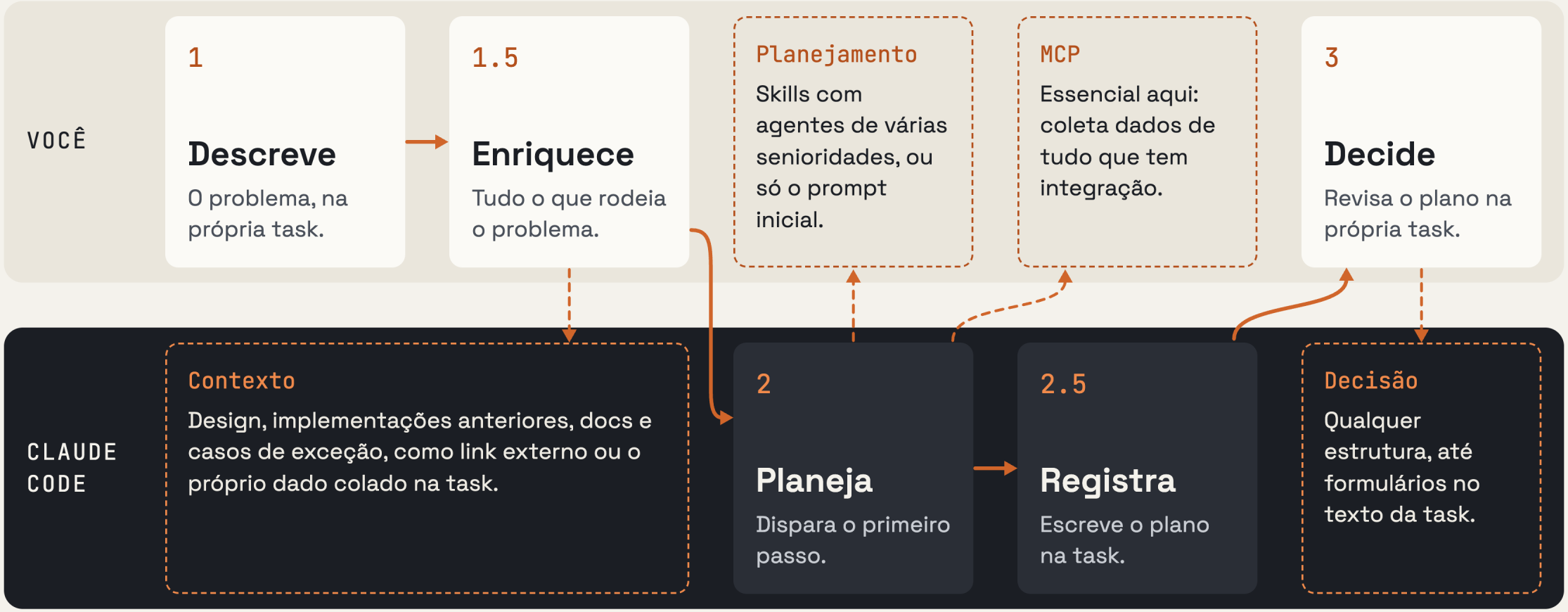
A task é o único lugar onde o problema vive.

`diagnóstico` → `planejamento` → `decisão` → `implementação` → `verificação` → `status`

Tudo registrado na task. Nada depende de chat, doc solto ou da memória de alguém.

O FLUXO

Do problema ao plano decidido



DOIS CONTEXTOS

Ferramentas diferentes, o mesmo padrão

	DanFit	Ponto Go
Ferramenta	Linear, tasks DAN-NN	ClickUp, saída em pt-BR (ID task)
Stack	iOS (SwiftUI, SwiftData, CloudKit), backends Go, front-ends (React), CLI interno	Go com gRPC e MongoDB, Next.js, CLI interno
Status	Planned → In Progress → In Review → Testing→ Production	Requisitos (Doc) → Design (Figma) → Task Enrichment → Planejamento → Plano Decidido → Implementação → Code Review
Elo com o código	Fixes DAN-NN fecha a task no merge na main	Gates de qualidade de implementação, análise estática do código.
Particularidade	Skills adicionais pra assertividade dos models registrados no CloudKit	Tem uma CLI para uso prioritário dos agentes (Database Checking). Criado para esse fluxo.

PARTE 2

A camada de skills

Instruções empacotadas e versionadas que o agente executa. O mesmo pipeline nos dois contextos.

`*-plan-main · *-reply · *-implement`

O PIPELINE

Três fases, três permissões

01

Planejar

`*-plan-main`

Diagnostica a causa raiz e escreve as opções na task.

Status → Planned ou gate de arquitetura

02

Decidir

`*-reply`

Você marca as escolhas. A skill relê o código e reescreve o plano.

Humano no loop

03

Implementar

`*-implement`

Segue o plano decidido, escreve testes, commita e atualiza a task.

Status → em progresso, depois revisão

`Código: somente leitura. A única escrita é na task.`

`Código: escrita liberada`

FASE 1 · PLANEJAR

Quatro lentes, uma causa raiz

Diagnóstico

Causa raiz lendo o código, antes de qualquer opção.

→

1 · Simples

A menor mudança cirúrgica possível.

2 · Arquitetural

Mudança estrutural. Sinaliza se precisa de ADR.

3 · Fora do escopo

Quando a resposta não é código: processo, produto, config ou outro repo.

4 · Balanceada

Pesa tudo junto e busca o caminho pragmático.

Saída na task: 4 opções, uma recomendada, checkboxes de decisão e perguntas em aberto.

Lentes isoladas (-simple, -architectural, -out-of-scope, -wide) quando só um ângulo interessa. No ClickUp, *-main-plan detecta o repo afetado e roteia para o planejamento certo.

FASE 2 · DECIDIR

Humano no fluxo, dentro da task

Task · Opções

- [ ] A · Simples
- [x] **B · Balanceada** (recomendada, marcada)
- [ ] C · Arquitetural
- [ ] D · Fora do escopo

Pergunta em aberto → sua resposta, escrita na task

→

Plano decidido

- Abordagem especificada
- Respostas incorporadas
- O que mudou em relação às opções
- Próximos passos e testes a criar

Ainda somente leitura no código. A decisão fica onde qualquer sessão futura vai procurar.

FASE 3 · IMPLEMENTAR

Só aqui o agente escreve código

1. Segue o plano decidido
2. Escreve os testes
3. Build e testes até ficar verde
4. Commit com a task linkada
5. Comenta o resumo na task
6. Move o status

No DanFit, a entrega é uma cadeia de skills

`gate`

Testes locais passando.

→

`merge-and-review`

Merge --no-ff com magic word, bump de versão, push, task em In Review.

→

`testflight-artifacts`

Build assinado, tag, tasks em On TestFlight.

ONDE O PLANEJAMENTO PARA

Plan e reply nunca tocam no código.

Não criam branch, não editam arquivo, não commitam. A única escrita delas é na task: texto e status.

A IA propõe e recomenda. A escolha é sempre humana.

A SEGUNDA FRONTEIRA

Nem a implementação executa o irreversível

O agente executa

- Código e testes
- Build e testes até ficar verde
- Commit e PR com magic word
- Resumo e status na task

O agente sinaliza e para

- Deploy
- Promoção de schema CloudKit em produção
- Drop de índice e backfill no Mongo
- Renomear permissões e mudar secrets
- Publicar na TestFlight sem opt-in explícito

Merge não encadeia deploy. O irreversível fica com um humano ou com um dono único, serializado pelo status.

MULTI-AGENTE · CLAUDE E CODEX

Dois agentes, um codebase, sem colisão

Divisão por task

Não por repo. Cada agente pega uma task inteira.

Status como lock

Reivindicar é mover para In Progress e se atribuir.

Worktree por task

Branch e diretório próprios, sem pisar no outro.

Dono único

Merge na main, bump, schema e TestFlight são serializados.

O gerenciador de tarefas vira mecanismo de coordenação, não só de registro.

VANTAGENS

O que muda na prática

Contexto persistente

Sessão nova, outro agente ou um colega retomam pela task, sem histórico de chat.

Rastreabilidade

Diagnóstico, opções, decisão, commit, PR e build, tudo linkado.

Opções antes do código

Comparar caminhos evita que a IA saia corrigindo sintoma.

Coordenação multi-agente

Status como lock e worktree por task, sem colisão.

Independe da ferramenta

Linear ou ClickUp; iOS, Go, Next.js ou CLI.

Memória institucional

ADRs e **gotchas** atualizados junto com a task quando sai uma decisão.

MEMÓRIA INSTITUCIONAL

ADRs e gotchas são arquivos no repo

docs/adr/0007-[titulo].md — ADR

```
# ADR-0007: [decisão]
Status: aceito
Contexto: o problema da task
Decisão: a opção marcada
Consequências: o que muda
Task: DAN-NN
```

docs/gotchas.md — Gotcha

```
## HealthKit
Sintoma: dados sincronizados, mas vazios
Causa: permissão revogada pelo usuário, não bug
no código
Task: DAN-NN
```

Markdown versionado, escrito pelo agente junto com a task quando sai uma decisão ou uma descoberta. O próximo planejamento parte dele.

POSSIBILIDADES

Casos que o mesmo fluxo cobre

Caso	O que o fluxo faz
Bug com causa raiz obscura	Diagnóstico antes do código. Dados ”sincronizados mas vazios” eram permissão revogada.
Feature em vários repos	Uma task gera vários PRs, todos linkados a ela.
Resposta que não é código	A lente fora do escopo aponta processo, produto ou config.
Mudança de schema ou dado	Gate manual no passo irreversível.
Épico grande	Subtasks com ordem de deploy definida.
Dois agentes em paralelo	Status como lock, worktree por task.

Caso Real

PARA LEVAR

A IA propõe.
Você decide.
A task lembra.

Perguntas?